

EBU6304 Software Engineering Group Project

2025-26

Group number	[Group Number]
member 1	QM no: 231224745 Name: Haoyang Li
member 2	QM no: 221170928 Name: Chuyuan Su
member 3	QM no: 231224365 Name: Yiyang Guo
member 4	QM no: 231224767 Name: Yizhou Ma
member 5	QM no: 221170799 Name: Lingran Qin
member 6	QM no: 231224918 Name: Changcheng Huang

First Report

1. Requirements

This section describes the fact-finding techniques, iteration plan, prioritisation, and estimation methods used in the International School Teaching Assistant Recruitment System project. The project follows Agile methodologies to develop a software application for BUPT International School to streamline the TA recruitment process.

1.1 Fact-Finding Techniques

To gather comprehensive requirements for the Teaching Assistant Recruitment System, our team employed multiple fact-finding techniques. These techniques were carefully selected to ensure we captured both the explicit needs and implicit expectations of all stakeholders involved in the TA recruitment process at BUPT International School.

1.1.1 Document Analysis

The team conducted a thorough analysis of existing documentation related to the current TA recruitment process. This included reviewing the project handout specification, existing application forms, Excel spreadsheets used for tracking applications, and any written procedures documented by the International School. The document analysis revealed that the current system relies heavily on manual processes using forms and Excel files, which creates inefficiencies in data management, tracking application status, and coordinating between multiple stakeholders. The analysis also identified key constraints, such as the requirement to use simple text file formats (TXT, CSV, JSON, XML) instead of databases, and the mandatory use of Java-based implementation without frameworks like Spring Boot.

1.1.2 Stakeholder Interviews

Structured interviews were conducted with representatives from each stakeholder group to understand their specific needs, pain points, and expectations from the new system. The interviews followed a semi-structured format, allowing for both predetermined questions and open-ended discussions. Key stakeholders interviewed included Module Organisers who post

TA positions and select candidates, Students who apply for TA positions, and Administrators who oversee the overall recruitment process and workload distribution. These interviews provided invaluable insights into the practical challenges faced by each user group and helped identify features that would deliver the most value.

1.1.3 Questionnaire Survey

A questionnaire survey was distributed to a broader audience of potential system users to gather quantitative data about user preferences and priorities. The survey was designed to capture information about the frequency of TA applications, common difficulties encountered in the current process, desired features for the new system, and preferences for user interface design. The survey received responses from students across different academic years and programmes, providing a diverse perspective on system requirements. The quantitative data from the survey helped validate findings from interviews and provided statistical support for prioritisation decisions.

1.1.4 Observation

Team members observed the current TA recruitment process in action, including how Module Organisers post positions, how students submit applications, and how administrators manage the overall workflow. This observational study revealed several inefficiencies not explicitly mentioned in interviews, such as the time-consuming nature of manual data entry, the lack of real-time status updates for applicants, and the difficulty in matching student skills with job requirements. These observations directly influenced the design of features like the AI-powered skill matching and the application status tracking system.

1.1.5 Prototyping and User Feedback

An iterative prototyping approach was adopted where low-fidelity and medium-fidelity prototypes were created and presented to stakeholders for feedback. This technique allowed stakeholders to visualise the proposed system and provide concrete feedback on the user interface design, workflow, and feature placement. The prototypes evolved through multiple iterations based on user feedback, ensuring that the final design aligns with user expectations and provides an intuitive user experience. This approach proved particularly valuable for refining the dashboard layouts for different user roles and the job application workflow.

1.2 Iteration Plan

The project follows an Agile development methodology with iterative sprints, allowing for incremental delivery, continuous feedback, and reflective improvement. The iteration plan is designed to deliver working software at regular intervals while maintaining flexibility to adapt to changing requirements and stakeholder feedback.

1.2.1 Sprint Structure and Timeline

The project is organised into four main iterations (sprints), each lasting approximately two to three weeks. Each sprint follows the standard Scrum framework with sprint planning, daily stand-ups, sprint review, and sprint retrospective meetings. The sprint structure ensures that the team maintains a sustainable development pace while delivering valuable functionality at the end of each iteration. The timeline aligns with the BUPT teaching calendar and scheduled lab sessions, allowing for regular interaction with the teaching assistant and demonstration of progress.

Table 1: Iteration Plan Overview

Iteration	Date Range	Key Deliverables
-----------	------------	------------------

Iteration	Date Range	Key Deliverables
Sprint 1	6 Mar - 22 Mar	Role selection, file storage system, profile management, CV upload functionality
Sprint 2	23 Mar - 12 Apr	Job posting (MO), job search/filter, job application with cover letter
Sprint 3	13 Apr - 3 May	Applicant selection (Accept/Reject), application status tracking
Sprint 4	4 May - 24 May	Admin dashboard, workload visualisation, alerts, AI skill matching

1.2.2 Sprint Goals and Feature Allocation

Each sprint has clearly defined goals that contribute to the overall project objectives. Sprint 1 focuses on establishing the foundational infrastructure, including the role-based access system and data storage mechanism, which are prerequisites for all subsequent features. Sprint 2 delivers the core functionality for job posting and application, enabling the primary workflow of the recruitment process. Sprint 3 completes the hiring loop by implementing the selection and status tracking features. Sprint 4 adds advanced features including administrative tools and AI-powered capabilities that enhance the system's value proposition. This incremental approach ensures that a minimum viable product is available early in the development cycle, with additional features being added progressively.

1.2.3 Daily Stand-ups and Communication

The team conducts daily stand-up meetings, either in person or virtually, to maintain transparency and address issues promptly. Each stand-up follows the standard format where team members share what they accomplished since the last meeting, what they plan to work on next, and any blockers they are facing. These meetings are kept brief (typically 15 minutes) to maintain focus while ensuring everyone is aligned on project progress. Additionally, the team uses GitHub for asynchronous communication, with issues, pull requests, and commit messages serving as documentation of ongoing work and decisions.

1.3 Prioritisation Methods

Effective prioritisation is crucial for delivering maximum value within the project constraints. The team employed multiple prioritisation techniques to ensure that the most important and valuable features are developed first, while managing dependencies and technical constraints.

1.3.1 MoSCoW Method

The MoSCoW method (Must have, Should have, Could have, Won't have) was used as the primary prioritisation framework. Each user story was categorised based on its importance to the core functionality of the system. 'Must have' features are essential for the system to function and deliver its primary value proposition - these include role selection, profile management, job posting, job application, and applicant selection. 'Should have' features are important but not critical for the initial release - these include application status tracking and workload visualisation. 'Could have' features are desirable but can be deferred if time constraints arise - these include advanced AI features like skill matching. 'Won't have' features are explicitly excluded from the current project scope but may be considered for future iterations.

1.3.2 Business Value Assessment

Each feature was assessed for its business value to the stakeholders. Features that directly

address the pain points identified during fact-finding were given higher priority. For example, the ability for Module Organisers to post jobs and select applicants was prioritised because it addresses the core need of streamlining the recruitment workflow. Similarly, features that benefit multiple stakeholder groups, such as the application status tracking that benefits both students and administrators, were given higher priority due to their broader impact.

1.3.3 Dependency Analysis

Technical dependencies between features were carefully analysed to determine the optimal implementation sequence. Features that serve as prerequisites for other features were prioritised earlier in the development cycle. For instance, the role selection and file storage system must be implemented before profile management, as the latter depends on both user authentication and data persistence. Similarly, job posting must precede job application, which in turn must precede applicant selection. This dependency-driven prioritisation ensures a logical development flow and minimises rework.

1.3.4 Risk-Based Prioritisation

Features with higher technical risk or uncertainty were prioritised earlier to allow more time for problem-solving and iteration. The file storage system, which is a mandatory requirement and involves technical constraints (no database, text file formats only), was given high priority to ensure the team has adequate time to design and implement a robust solution. Similarly, AI-powered features, while lower in overall priority, were scheduled for earlier sprints to allow time for experimentation and refinement based on the project handout's guidance on responsible AI integration.

1.4 Estimation Methods

Accurate estimation is essential for effective sprint planning and resource allocation. The team employed a combination of estimation techniques to derive realistic effort estimates for each user story.

1.4.1 Story Points and Fibonacci Scale

User stories were estimated using story points on a Fibonacci scale (1, 2, 3, 5, 8, 13, 21). The Fibonacci scale was chosen because it naturally accounts for increasing uncertainty with larger tasks - as tasks become more complex, the gaps between estimates widen, reflecting the greater uncertainty in estimation. A story point value of 1 represents a trivial task that can be completed in a few hours, while 8 represents a complex task that may take several days. Stories estimated at 13 or higher were typically broken down into smaller, more manageable stories.

1.4.2 Planning Poker

Planning poker sessions were conducted to achieve consensus on story point estimates. During these sessions, each team member independently estimated the effort required for a user story, then revealed their estimates simultaneously. This approach prevents anchoring bias where early estimates influence subsequent estimates. When estimates differed significantly, team members discussed their reasoning, considering factors such as complexity, amount of work, and risk or uncertainty. This collaborative estimation process led to more accurate estimates and ensured shared understanding of the work involved.

1.4.3 Reference Stories

The team established reference stories to serve as baselines for estimation. A reference story

with 3 story points was defined as a typical feature implementation involving basic UI, simple business logic, and file I/O operations. New stories were then estimated relative to this reference. For example, a story estimated at 5 points would be considered slightly more complex than the reference, perhaps involving additional validation logic or more complex UI interactions. This relative estimation approach improved consistency across the team's estimates.

1.4.4 Velocity Tracking

The team tracks velocity (the number of story points completed per sprint) to improve future estimation accuracy and sprint planning. By monitoring actual velocity against planned velocity, the team can adjust its capacity estimates and identify patterns in estimation accuracy. This empirical data helps the team make more realistic commitments in sprint planning and provides early warning if the project is at risk of missing deadlines.

2. Supporting Material

This section presents the supporting materials gathered during the requirements elicitation process, including interview questions, survey results, and customer feedback that informed the development of the product backlog.

2.1 Interview Questions

The following interview questions were designed to gather detailed requirements from each stakeholder group. The questions were structured to explore current processes, pain points, and desired improvements.

2.1.1 Questions for Module Organisers

1. How do you currently post TA positions? What information do you typically include?
2. What challenges do you face when reviewing TA applications?
3. What criteria do you use to select TAs? How do you assess candidate suitability?
4. How do you currently communicate decisions to applicants?
5. What information would help you make better hiring decisions?
6. How many TA positions do you typically need per semester?
7. What specific skills or qualifications do you look for in TAs?

2.1.2 Questions for Students (TA Applicants)

1. How do you currently find out about TA opportunities?
2. What difficulties have you experienced when applying for TA positions?
3. What information would you like to see about available positions?
4. How would you prefer to track the status of your applications?
5. What skills or experiences would you want to highlight in your profile?
6. How many TA positions do you typically apply for per semester?
7. What would make the application process easier for you?

2.1.3 Questions for Administrators

1. How do you currently monitor the overall TA recruitment process?
2. What challenges exist in managing TA workload distribution?
3. How do you identify potential overallocation of TAs?
4. What reports or metrics would help you manage the process better?
5. What are the maximum working hours allowed for TAs?
6. How do you currently handle conflicts in TA assignments?

2.2 Survey Results

We have not yet conducted the questionnaire survey, so there are no survey results or customer feedback.

2.3 Customer Feedback

We have not yet conducted the questionnaire survey, so there are no survey results or customer feedback.